“SMART ALARM CLOCK”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

DARSHINI GANGISETTI	**2520030167**
PATURU MADHULIKA	**2520030151**
V.CHAITANYA	**2520039619**

Under the Esteemed Guidance of

Dr. E.Soumya
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

The Project Report entitled **"Smart alarm clock"** is a record of Bonafide work of DARSHINI GANGISETTI-2520030167, PATURU MADHULIKA -2520030151,V.CHAITANYA-2520039619 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/ University/Institute.

DARSHINI GANGISETTI	**2520030167**
PATURU MADHULIKA	**2520030151**
V.CHAITANYA	**2520039619**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled**"Smart alarm clock"** is a bonafide work done and submitted by DARSHINI GANGISETTI-2520030167, PATURU MADHULIKA -2520030151,V.CHAITANYA-2520039619 in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide Dr.E.Soumya**,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

DARSHINI GANGISETTI **2520030167**
PATURU MADHULIKA **2520030151**
V.CHAITANYA **2520039619**

Abstract

The **Alarm Clock** project is a Linux-based system programming application that allows users to set an alarm for a specific time and receive an alert when the set time is reached. The project demonstrates important Operating Systems and System Programming concepts such as process management, signals, time management, and system calls. The application will be developed using **C programming in Linux/Ubuntu** and will use POSIX/Linux APIs to interact with the operating system. The program will obtain the current system time, accept the required alarm time from the user, and continuously monitor the time until the alarm condition is satisfied. Once the specified time is reached, the program will generate an alarm notification. The project helps demonstrate how a user-level application interacts with the Linux operating system and how system resources such as time and processes are managed. The expected outcome is a functional command-line alarm clock that successfully sets and triggers alarms while demonstrating practical Operating Systems concepts.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgemen		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background		6–12
		2.1 Operating System Concepts Related to the	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	

5	**System Architecture and Design**		**23–29**
		5.1 Overall System Architecture	
		5.2System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	
		5.5 Input and Output Design	
6	**Implementation**		**30–45**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**62–67**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**79–83**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**84–86**

		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Time management is an important part of a student's daily life. Students need to manage classes, study sessions, assignments, meetings, personal activities and deadlines. A traditional alarm clock only provides an alert at a particular time. It does not provide information about what activity should be performed at that time.

The **Smart Alarm Clock** is designed to overcome this limitation by combining alarm management with activity scheduling and task management. Instead of simply ringing at a specified time, the system connects an alarm with a meaningful activity such as studying, attending a meeting, completing an assignment or following a daily routine.

The project is implemented using **C programming and Linux/POSIX system-programming concepts**. It operates through a terminal-based interface and provides features such as alarms, reminders, daily schedules, meeting rescheduling and prioritized to-do tasks.

1.2 Problem Statement

Basic alarm clocks mainly provide time-based alerts. They do not provide an integrated way to manage activities associated with those alarms.

Students may use separate applications for alarms, calendars and to-do lists. This makes it difficult to manage all daily activities from one place.

Therefore, the problem addressed by this project is:

“To develop a smart, terminal-based alarm and productivity system that connects scheduled alarms with activities, reminders, meetings and prioritized tasks.”

1.3 Scope of the Project

The scope of the Smart Alarm Clock includes:

- Creating and managing alarms.
- Associating alarms with specific activities.
- Scheduling activities according to date and time.
- Creating study and task reminders.
- Viewing upcoming activities.
- Displaying a daily overview.
- Marking activities as completed or missed.
- Creating group-meeting reminders.

- Rescheduling meetings.
- Maintaining a prioritized to-do list.
- Supporting repeated alarms.
- Providing snooze functionality.
- Running the complete application through the Linux terminal.

1.4 Objectives

The main objectives of the project are:

1. To develop a time-based alarm system using C.
2. To connect alarms with meaningful daily activities.
3. To provide a daily schedule and upcoming-activity view.
4. To support study, assignment and meeting reminders.
5. To allow activities to be marked as completed or missed.
6. To provide group-meeting rescheduling.
7. To maintain a prioritized to-do list.
8. To demonstrate Linux and POSIX system-programming concepts.
9. To provide a simple terminal-based productivity application.

1.5 Importance of the Application

The application converts a basic alarm clock into a small productivity-management system.

For example, instead of setting an alarm only for **9:00 AM**, the user can create an activity such as:

9:00 AM → Study DSA → High Priority

This makes the alarm more meaningful and helps users organize their daily routine.

1.6 Target Users

The system can be useful for:

- Students.
- People managing daily routines.
- Users managing study schedules.

- Users who need meeting reminders.
- Users who want a simple task-management system.
- Students learning C and Linux system programming.

OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts

The Smart Alarm Clock demonstrates several operating-system-related concepts.

Time Management

The application continuously works with the system date and time to determine when an activity should be triggered.

Process Execution

The Smart Alarm Clock runs as a user-space process in Linux.

Terminal Input/Output

The user interacts with the application through the Linux terminal using keyboard input and terminal output.

Scheduling

Activities are organized according to their scheduled date and time.

System Time

The application obtains the current system time and compares it with scheduled activities.

POSIX Programming

Linux/POSIX time-related functions are used for date and time processing.

2.2 Existing Systems

Existing System	Function	Limitation
Basic Alarm Clock	Provides time-based alarms	No activity management
Mobile Alarm	Alarm, snooze and repeat	Productivity features may be separate
To-do List	Stores tasks	Does not necessarily trigger alarms
Calendar	Manages events	Can be more complex than required

2.3 Proposed System

The proposed Smart Alarm Clock combines:

Alarm + Activity + Reminder + Meeting + To-do Management

in a single terminal-based application.

An activity can contain information such as:

- Activity ID
- Activity title
- Scheduled date
- Scheduled time
- Repeat option
- Snooze duration
- Activity status
- Meeting information

The to-do system maintains:

- Task ID
- Task title
- Priority
- Completion status
- Due date/time

2.4 Advantages

- Simple terminal-based interface.
- Combines alarms and tasks.
- Supports daily activities.
- Supports repeated reminders.
- Supports meeting rescheduling.
- Supports task priorities.
- Provides activity status.
- Demonstrates practical C programming.

CHAPTER – 3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Minimum Requirement
Processor	Modern 64-bit processor
RAM	4 GB
Storage	300 MB free space
Display	1280 × 720

3.2 Software Requirements

- Linux / Ubuntu
- C Programming Language
- GCC Compiler
- POSIX APIs
- VS Code or any text editor
- Linux Terminal

3.3 Operating System Environment

The project is designed to run in a **Linux/Ubuntu environment**.

The Linux terminal is used for:

- Writing/compiling the program.
- Running the executable.
- Providing input.
- Displaying alarms and tasks.
- Testing the application.

3.4 Development Tools

Tool	Purpose
C	Main programming language
GCC	Compilation
Linux/ Ubuntu	Execution environment
POSIX	System programming
VS Code	Code editing
Terminal	Execution and testing

CHAPTER – 4 TECHNOLOGY AND TOOLS USED

4.1 C Programming

C is used as the primary programming language.

The program uses:

- Structures
- Arrays
- Functions
- Conditional statements
- Loops
- String handling
- Date/time handling
- File/data handling

Structures are particularly useful for representing alarms and to-do tasks.

4.2 GCC Compiler

GCC is used to compile the C source code.

A typical compilation command is:

```
gcc -std=c11 -Wall -Wextra smartalarmclock.c -o smartalarmclock
-lm
```

The generated executable can then be executed using:

```
./smartalarmclock
```

4.3 Linux / POSIX

Linux provides the environment in which the application runs.

POSIX/Linux time-related facilities are used for handling scheduled activities and current time.

The implementation uses concepts such as:

- `time_t`
- `struct tm`
- `localtime_r()`
- `mktime()`

4.4 Data Structures

Alarm Structure

The alarm record contains information such as:

- ID
- Title
- Participants
- Meeting status
- Scheduled time
- Repeat option
- Snooze duration
- Active status
- Activity status
- Last fired time

To-do Structure

The to-do record contains:

- ID
- Title
- Priority
- Completion status
- Due date
- Due-time information

4.5 Priority Levels

The system uses:

1. **Critical**
2. **High**
3. **Medium**
4. **Low**

This allows important tasks to be identified quickly.

CHAPTER – 5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The Smart Alarm Clock can be divided into the following logical modules:

1. User Interface

Provides the terminal menu and accepts user input.

2. Alarm Manager

Handles creation, viewing, updating and deletion of alarms.

3. Activity Manager

Associates scheduled times with meaningful activities.

4. Meeting Manager

Handles group meetings and rescheduling.

5. To-do Manager

Maintains tasks and their priorities.

6. Time Monitor

Checks the current system time against scheduled activities.

7. Persistence

Stores and loads activities and tasks.

5.2 System Workflow

The overall workflow is:

START

↓

Load saved data

↓

Get current system time

↓

Display main menu

↓

User selects operation

↓

Create / View / Update / Delete activity

↓

Monitor scheduled time

↓

Generate reminder

↓

User completes / snoozes / reschedules

↓

Update status

↓

Save data

↓

Exit

5.3 Main Functional Modules

Module	Function
Alarm Module	Create and manage alarms
Activity Module	Manage scheduled activities
Meeting Module	Create/reschedule meetings
Reminder Module	Repeated reminders and snooze
To-do Module	Manage prioritized tasks
Overview Module	Display daily schedule
Persistence Module	Save/load information

5.4 Main Menu Operations

The application provides operations such as:

1. Add alarm/activity
2. View all alarms
3. View upcoming activity
4. Daily overview
5. Update activity status
6. Delete alarm/activity
7. Reschedule group meeting
8. Save and exit
9. Add to-do task
10. View prioritized to-do list
11. Mark to-do task completed
12. Delete to-do task

CHAPTER – 6 IMPLEMENTATION

6.1 Alarm and Activity Creation

The user provides information such as:

- Activity name
- Date
- Time
- Repeat option
- Snooze duration
- Meeting information where required

The system validates the information and creates the activity.

6.2 Time Handling

The system uses date and time structures to represent scheduled activities.

The current system time is obtained and compared with the scheduled activity time.

Functions such as:

`localtime_r()`
`mktime()`
are used for date/time processing.

6.3 Alarm Monitoring

The system continuously monitors scheduled activities.

When the current time reaches the scheduled time:

Scheduled Time = Current Time

the corresponding reminder becomes due.

The activity can then be completed, snoozed or otherwise updated according to the available options.

6.4 Repeated Alarms

The system supports daily repetition.

For example:

Activity: Study DSA
Time: 9:00 AM
Repeat: Daily

The activity can therefore be treated as a recurring reminder.

6.5 Snooze Function

The snooze option allows the user to postpone an alarm for a specified number of minutes.

Example:

Alarm: 9:00 AM
Snooze: 5 minutes

The next reminder can occur at:

9:05 AM

6.6 Group Meeting Rescheduling

The project includes a dedicated meeting-rescheduling operation.

The user:

1. Views the available activities.
2. Selects the meeting ID.
3. Confirms that it is a group meeting.
4. Enters a new date.
5. Enters a new 24-hour time.
6. The system validates the new time.
7. The meeting is updated.

6.7 To-do Management

The user can create tasks such as:

- Study
- Assignment
- Project work
- Meeting preparation
- Personal activity

Each task can be assigned a priority.

Example:

Task	Priority
Complete assignment	Critical
Study DSA	High
Read notes	Medium
Organize files	Low

6.8 Activity Status

Activities can have different states such as:

- Pending
- Completed
- Missed

This allows the daily overview to provide more information than a simple alarm list.

6.9 Persistence

The application saves activities and tasks so that the information can be retained and reused.

This prevents the user from having to recreate every activity each time the application is started.

CHAPTER – 7

EXECUTION AND DEMONSTRATION

7.1 Compilation

Open the Linux terminal and move to the project directory.

Compile using:

gcc -std=c11 -Wall -Wextra smartalarmclock.c -o smartalarmclock -lm
If compilation is successful, execute:

./smartalarmclock

7.2 Application Start

When the application starts, it displays:

- Application title
- Current date/time
- Existing activities/tasks
- Main menu

The user can select the required operation by entering its number.

7.3 Sample Activity

Example:

Activity: Study DSA
Date: 2026-09-29
Time: 09:17
Repeat Daily: Yes
The system stores the activity and monitors its scheduled time.

7.4 Sample To-do Tasks

Example:

```
1. Study DSA        Priority: HIGH
2. Complete work    Priority: CRITICAL
3. Eat              Priority: MEDIUM
```

The system displays tasks according to their priority.

7.5 Testing

Test Case	Input	Expected Result

Create activity	Valid date/time	Activity created
Daily repeat	Repeat = Yes	Daily reminder enabled
Add task	Task + priority	Task added
View tasks	To-do option	Prioritized list displayed
Complete task	Task ID	Task marked completed
Reschedule meeting	Meeting + new time	Meeting updated
Upcoming activity	View upcoming	Future activities displayed
Daily overview	Daily overview	Schedule/status displayed

7.6 OS Concepts Demonstrated

OS Concept	Project Demonstration
Process	Application runs as a Linux process
System time	Current time used for alarms
Scheduling	Activities arranged by scheduled time
Terminal I/O	User interacts through terminal
POSIX	Time-related system interfaces
Persistence	Data is stored and retrieved
Resource execution	Program executes using Linux environment

CHAPTER – 8 CONCLUSION

8.1 Summary

The **Smart Alarm Clock** is a terminal-based time-management and productivity system developed using C and Linux/POSIX concepts.

Unlike a conventional alarm clock, the application connects alarms with meaningful activities. It provides features for managing activities, reminders, meetings and prioritized tasks.

The project demonstrates how basic operating-system and programming concepts can be combined to develop a practical application.

8.2 Objectives Achieved

The project successfully provides:

- Alarm management.
- Activity scheduling.
- Daily overview.
- Upcoming activity view.
- Repeated reminders.
- Snooze functionality.
- Group meeting reminders.
- Meeting rescheduling.
- Prioritized to-do list.
- Task completion.
- Activity status management.
- Linux terminal execution.

8.3 Results

The application demonstrates successful creation and management of activities and tasks.

The project execution shows:

- Current time display.
- Activity creation.
- Daily repetition.
- To-do task creation.

- Priority-based task display.
- Meeting rescheduling functionality.
- Daily activity management.

8.4 Learning Outcomes

Through this project, the following concepts were practiced:

- C programming.
- Structures and arrays.
- Functions and modular programming.
- Linux terminal usage.
- GCC compilation.
- POSIX programming.
- Date/time handling.
- Scheduling logic.
- File/data persistence.
- Task prioritization.
- Software testing.

8.5 Limitations

The current system is primarily terminal based. It does not provide a graphical user interface or mobile application interface.

Advanced features such as online calendar synchronization and cloud-based notifications are also outside the current implementation.

8.6 Future Enhancements

The system can be improved by adding:

1. Graphical user interface.
2. Mobile application.
3. Desktop notifications.
4. Custom alarm sounds.
5. Voice reminders.
6. Calendar synchronization.

7. Advanced recurring schedules.
8. Cloud data storage.
9. Multi-user meeting management.
10. Network-based reminders.

The project can therefore be further developed into a complete personal productivity and scheduling platform.

CHAPTER – 9 REFERENCES

1. Supplied Smart Alarm Clock project presentation and implementation material.
2. Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*.
3. Linux/GNU C Library documentation for time and date handling.
4. POSIX programming documentation.
5. GCC compiler documentation.
6. Linux system-programming references.